

become zero. This leaves us with the only possibility that each of the elements in (x,y,z) was constructed from a different set. Hence, we have representatives constructed from all three sets A , B and C in (x,y,z) . Therefore, we have shown that if we can solve PR1 on the set S that we have constructed, we have a solution for PR2 also.

Now we can show a $O(N^2)$ algorithm for solving PR2 as follows: we sort the sets B and C . Consider an element $a \in A$. Now consider the set $B + a$ (the set of all numbers in B such that each number has a added to it). The set $(B + a)$ can be computed in $O(N)$ time. We now traverse the sets $(B + a)$ and C simultaneously to check if there is a common element between $(B + a)$ and C . This can be done in $O(N)$ time since these sets are sorted.

If we find a common element, we have found the triplet (a, b, c) such that $a + b = c$. Else we repeat the above procedure with the next element in A . Since we may end up repeating the above procedure for each of the elements in A , we end up with an overall complexity of $O(N^2)$ for our solution.

Now let us turn our attention to this month's topic of computational complexity and lower bounds for algorithms.

Computational complexity

Let us start with a simple children's game. You and your son are playing a number game. You tell your son that he can think of any positive number between zero and one million. You will ask him 20 questions to which he has to answer either "Yes" or "No" truthfully. You challenge your son that at the end of the 20 questions, you will be able to tell your son what number he had originally thought of. Your son is amazed at your ability to read his mind. Now how do you manage to answer correctly? The answer, of course, lies in the divide and conquer approach. With each question, you reduce the number range where the mystery number can possibly lie. For instance, your first question to your son is: "Is the number between 1 and 5,00,000?" An answer of "Yes" reduces your working range from 1 to 5,00,000, whereas an answer of "No" will reduce the answer range to 5,00,001 to one million. You know that you can always find the answer within your allotted 20 questions because one million is less than 2^{20} .

Okay, now that you have enthralled your son with your abilities, he asks you a question: Do you need 20 questions to arrive at the correct answer or can you do it with fewer questions? Now you are stumped. You know that the binary search algorithm that you have used has an upper bound of $O(\log N)$, and therefore you know that 20 questions are sufficient to solve the problem. But your son's question to you now is whether 20 questions are necessary or can you do this with a fewer number of questions no matter what number he thinks up. This is essentially the realm of the computational complexity of problems. For this problem of finding a number in a given range, we can show that the computational complexity

has a lower bound of $O(\log N)$. That is, no matter what the algorithm you or anyone can else come up with, it will have a lower bound on the complexity as $O(\log N)$. If you had known this fact, you can tell your son that 20 questions are, in fact, necessary, for any arbitrary number he can think of. If you decide to challenge him to a context where only 19 questions are allowed, he can always win. I leave the detailed proof of this statement to interested readers since it would be fun to try this game with a friend/family member a few times first before trying to prove the statement rigorously.

Lower bounds of some well-known computational problems

Complexity analysis tries to prove that for a given problem, any algorithm capable of solving the problem correctly on all of its instances, must necessarily take a time that is greater than or equal to the established lower bounds for that problem. Basically, computational complexity theory establishes that there cannot exist an algorithm that can solve the problem in a time lower than the established lower bound, under the specified model of computation. For instance, it is widely known that any sorting algorithm based on comparison for sorting an array of N numbers has a lower bound of $O(N \log N)$. This means that there can exist no sorting algorithm based on the comparison of elements, which can sort faster than this.

There are a number of ways of establishing the lower bounds for any problem. Two of the well-known methods are the decision tree method and adversary argument. The decision tree method is well known and the interested reader can refer to any algorithm book for details on this. Let us take a look at the adversary argument for establishing lower bounds. Let us consider the well-known and simple problem of finding the maximum, given an array of N numbers.

Given below is the code snippet to find the maximum of a given set of N numbers, using only comparison between the numbers:

```
int find_maximum(int array[], int N)
{
    int max = A[0];

    for (int i = 0; i < N; i++)
    {
        if (a[i] > max)
            max = a[i];
    }

    return max;
}
```

It is evident that the algorithm given above uses exactly $(N-1)$ comparisons to arrive at the answer, since the *for* loop runs for $N-1$ iterations. Hence the algorithmic